ArchIntel — User Manual

Phase 6 checkpoint: Controlled source tasks and the pinned local Docker build sandbox are implemented. A controlled Java/Maven fixture passed a real offline, non-root, network-disabled build and test. Full Phase 6 acceptance remains open: customer artifact reproduction and broader characterization/security/lineage capabilities remain pending. See verification, user flow and developer handoff. Phase 7C now discovers supported source data schemas/access paths, generates target-schema and mapping proposals without requiring a customer-designed target, and runs approved synthetic fixtures in an ephemeral SQLite database. Production migration and database-specific recovery validation remain disabled and open. See the Phase 7C user flow, developer handoff and verification.

Phase 5 update: The Modernization intelligence workspace now creates immutable evidence-backed planning proposals, separate readiness/confidence, decisions, targets and dependency waves, with validation and actor-attributed review. Planning approval does not authorize code changes. See user flow, developer guide and verification.

Phase 4 evidence intelligence (4 October 2026): Open Evidence & digital twin for version capture, reviewed domains, fact provenance, guided discovery and the persistent knowledge graph. See user flow, developer handoff and validation. Existing static/runtime workflows remain available.

Phase 3 runtime update (4 October 2026): For current session setup, instrumentation preview/exclusions, strict identity declarations, quality diagnostics and recovery, read Phase 3 user flow and developer handoff. Validation results include a new real PetClinic capture; older results below remain historical.

Optional local-model setup and controls: Phase 2 user flow. Internal contracts and extension rules: Phase 2 developer guide. **Version 0.2 | Phase 2 AI gateway | 4 October 2026**

What the tool does

ArchIntel inspects a compiled Java application and produces architecture graphs, candidate domains, technical-debt findings, modernization readiness, and a suggested investigation

sequence. The uploaded application is never launched. Results describe static structure, not proven production behavior.

Northstar Commerce is a **synthetic demonstration application**. Its results are generated by the same real jQAssistant → Neo4j → architecture-engine pipeline used for uploads.

Start and stop

On the prepared Windows machine:

1. Open this project folder and double-click **Start-ArchIntel.cmd**.
2. Wait for “ArchIntel ready.”
3. The browser opens **http://127.0.0.1:8000**.
4. Complete administrator setup or sign in, then confirm **Services ready** in the top bar. See Phase 1 user flow for the setup key and access controls.

Open **Local setup** in the left navigation for environment diagnostics. This page checks the configured database, local Java static-analysis runtime, and ArchIntel data folder. It distinguishes optional runtime-agent/local-model setup from container-based candidate build and security workflows, which are deferred in the local POC. Docker is not needed for ordinary static analysis or for receiving runtime telemetry; runtime capture also requires an OpenTelemetry Java agent in the application being observed.

The launcher starts workspace-local PostgreSQL and the API in hidden background processes. The prepared runtime does not require Docker or an external API key. Internet is needed for initial setup, not the prepared demonstration.

After a reboot, run the launcher again. To stop, wait for analyses to finish and double-click **Stop-ArchIntel.cmd**. Saved results remain in `.data/postgres`; do not delete that folder.

Investor walkthrough: 7–10 minutes

Step	Screen	What to demonstrate
1	Applications → Explore demo application	Explain that the sample is synthetic code scanned through the real pipeline.
2	Overview	Review 40 classes, seven packages/domains, dependencies, and issues.
3	Architecture explorer	Start with domains; expand packages and classes.
4	Circular dependencies	Compare a cross-domain cluster with a same-package reporting cycle.

Step	Screen	What to demonstrate
5	Technical debt	Filter severity and expand a finding to inspect its evidence.
6	Candidate domains	Open Payment and inspect the proposed boundary and database signals.
7	Modernization plan	Open a row to reveal score components, risks, and dependency waves.
8	Architecture assistant	Ask “Why is Payment high risk?” and open a citation.
9	Export report	Download the full structured analysis.

The demo button reopens a previously completed real scan when available. To show fresh processing, download the sample JAR and upload it as a new application. Keep one completed analysis ready before the meeting: starting Java and reading a graph store takes time.

Upload your own application

1. Select **Applications**, then **Analyze application**.
2. Choose a project or enter a new project name.
3. Enter an application name and version.
4. Choose a compiled `.jar` or `.war`.
5. Click **Start architecture analysis**.
6. Follow progress until the overview appears.

Supported packaging: ordinary JARs, WAR classes under `WEB-INF/classes/`, and Spring Boot classes under `BOOT-INF/classes/`. Nested dependency JARs are not recursively scanned. Application references to external libraries are retained separately. Multi-release override classes are skipped; base classes are analyzed.

Default safety limits: 100 MB uploaded, 400 MB expanded, 50,000 entries, 20,000 classes, and 200,000 extracted dependency rows. These are protective limits, **not a measured large-application capacity guarantee**.

Unsafe paths, malformed archives, encrypted entries, symbolic links, duplicate normalized class paths, and excessive expansion are rejected. Upload artifacts are removed after successful processing by default. Failed inputs are retained for retry; structured results and the artifact hash are retained.

Processing progress

The stages are validation, scanning, graph extraction/building, cycle analysis, metrics, domain discovery, issue analysis, modernization, and result persistence. Each stage records duration. A failed run reports the failed stage and never presents partial findings as completed results.

One job executes at a time. Up to four jobs can be queued/running. Refreshing the browser does not cancel a run. On startup, queued/interrupted work restarts from its original artifact when available; missing inputs produce an actionable recovery failure.

Overview

The overview shows filtered application classes/packages, distinct internal dependency pairs, external references, candidate domains, issues, and average structural readiness. It also identifies the artifact SHA-256, scanner, and algorithm version.

A pair of classes can have several dependency types. Consequently, the typed-edge count may exceed the graph's distinct-pair count. Average readiness is not a forecast of migration cost, duration, or success.

Architecture explorer

- Switch between **Domains**, **Packages**, and **Classes**.
- Click a node for details; double-click a domain/package to expand it.
- Use the domain filter to narrow a large view.
- Drag nodes, pan, scroll to zoom, and use the lower-right fit button.
- Clear the package filter to broaden a class view.

Arrows point **from the dependent to its dependency**. Views are capped at 150 displayed nodes and 2,000 edges; a notice identifies truncation. Edges require both endpoints to be visible. Class and package graphs remain separate: a package cycle can exist without a class cycle.

Candidate domains

Cards show names, class/package counts, heuristic confidence, outgoing cross-domain ratio, issues, and readiness. Open a card for grouping evidence, database signals, dependency counts, and class membership.

Confidence is not a probability of business correctness. Validate boundaries and naming with domain owners. Current boundaries are automatically generated; manual editing is a future extension.

Technical debt and cycles

Filter issues by severity, type, domain, or search text/package. Expand a finding to inspect its affected entity, evidence, significance, and recommended investigation.

Rules cover circular dependencies, large cyclic clusters, cross-domain coupling, god-class candidates, unstable-package dependencies, main-sequence deviation, high fan-in/out, and boundary violations. A god-class candidate requires multiple relative outlier signals, not method count alone.

Cycle cluster membership is complete; displayed cycle paths are bounded samples. Large clusters skip enumeration while retaining membership and internal edge counts. No short sampled cycle does not mean a strongly connected cluster is acyclic.

Architecture metrics

Metric	Meaning
Ca	Distinct outside classes that depend on a package's classes.
Ce	Distinct outside classes depended on by a package's classes.
I	Instability: `Ce / (Ca + Ce)`; zero when both are zero.
A	Abstract classes or interfaces / all package classes, counted once per type.
D	Distance: `abs(A + I - 1)`.

Interpret instability with abstractness and architectural role. Neither high nor low instability is inherently good or bad. Isolated packages are identified separately. The scatter plot's diagonal is the main sequence; hover points for values.

Modernization plan

Readiness is a 0–100 weighted structural score. Open a row to inspect the actual component values and configured weights. Defaults: cohesion 25%, decoupling 30%, cycle independence 25%, database evidence 20%.

Unknown database evidence receives a neutral component and remains an explicit risk. Its displayed range is sensitivity analysis, not statistical confidence. The tool does not infer runtime SQL, transactions, table ownership, or a data migration plan.

The sequence is **dependency first**: prerequisites appear in earlier waves. Readiness prioritizes within a wave. Domains in the same cyclic group require coordinated remediation; their row order is

not an independently executable extraction plan.

Architecture assistant

Example questions:

- “Where should modernization begin?”
- “Why is Payment high risk?”
- “Which domains are involved in cycles?”
- “Give me an architecture overview.”

The prepared mode uses **local deterministic evidence explanations**, without a generative model or external transmission. Facts and recommendations are separate. Click a citation to inspect its analysis-owned source. Unsupported questions receive an evidence-limited response.

An optional local-model adapter can select relevant citation IDs; it cannot introduce arbitrary new facts. Do not represent the default mode as a live generative LLM.

Export and manuals

Export report downloads JSON containing graphs, metrics, domains, issues, cycles, readiness components, sequencing, configuration, limitations, and artifact identity. **Help & documentation** provides this manual and the developer guide. Printable HTML/PDF copies are in `docs`.

Troubleshooting

Symptom	Action
Cannot connect	Run the launcher and use port 8000, not an old development tab.
Services unavailable	Inspect `.data/api.err.log` and `.data/postgres.err.log`; retry the launcher after startup.
Scanner runtime incomplete	Follow setup/exporter-build instructions in the developer guide.
Upload rejected	Confirm a compiled JAR/WAR within the archive limits.
No internal classes remain	Have a developer review `EXTERNAL_PREFIXES` for your application's namespace.
Queue full	Wait for an existing run to finish.

Symptom	Action
Run interrupted by restart	Restart ArchIntel. Queued/interrupted runs resume automatically when their original artifact is available. If recovery reports a missing artifact, upload it again.
Scan failed	Correct the reported cause and start analysis again for the application. The failed input is retained for retry; the previous failed run remains in history. A checksum failure requires uploading the original artifact again.
Browser origin is not allowed	Open the tool directly at its localhost URL. A developer must explicitly configure trusted origins and hosts for a different deployment.
Question cannot be answered	Ask about a named domain or supported structural finding.

Prototype boundaries

This is a local prototype with static analysis, OTel runtime analysis, authenticated workspaces and a read-only source security gateway (see `RUNTIME-USER-FLOW.md` and `PHASE-1-USER-FLOW.md`). Production deployment hardening, automatic rewriting, billing and generated microservices remain future work. Keep it bound to localhost. Validate runtime behavior and business context before acting on modernization recommendations.

Failed scan artifacts are currently retained until a successful retry processes them or an operator removes them. Successful processing deletes the uploaded artifact unless `RETAIN_ARTIFACTS=true`. Automatic age-based failed-artifact retention is still planned.